

PRODUCT REQUIREMENTS DOCUMENT

Sutradhar — Product Requirements Document

A production-grade, open-source real-time Voice AI agent platform

Document type	Product Requirements Document (PRD) + Technical Design
Version	1.0
Status	Approved for build
Date	26 June 2026
Author	Reeth Jain (Founding AI Engineer)
Codename	<i>Sutradhar</i> — Sanskrit for the orchestrator who "holds the strings"
Target reviewers	Engineering hiring managers at Voice AI companies (Sarvam AI, Gnani.ai, Bolna AI, Ring AI, Riverline, SquadStack)

Table of Contents

1. Executive Summary

Hero differentiators

2. Problem Statement & Motivation

Why this is a strong resume artifact

3. Goals & Non-Goals

3.1 Goals

3.2 Non-Goals

4. Target Users & Personas

5. Product Overview & Vision

6. System Architecture

6.1 Architectural principles

6.2 Layered view

6.3 Core interfaces

7. Technology Stack (tuned for GTX 1650 / 4 GB VRAM / 16 GB RAM)

8. Latency Budget & Performance Requirements

Performance requirements

9. Turn-Taking & Barge-In (Hero Feature #1)

9.1 Endpointing (when did the user stop?)

9.2 Barge-in (user interrupts the agent)

9.3 Turn states

10. Conversation State, Memory & Tool-Calling

11. Evaluation Framework (Hero Feature #3)

12. Observability & Monitoring

13. Fault Tolerance & Reliability (Hero Feature #2 supports this)

14. Testing Strategy

15. Deployment & Developer Experience

16. Repository Structure

17. Roadmap & Milestones

18. Functional Requirements

19. Non-Functional Requirements

20. Risks & Mitigations

21. Success Metrics / KPIs

22. Future Work (commercial path)

23. Appendix

A. Glossary

B. Default model table

C. Key references / inspiration

1. Executive Summary

Sutradhar is an open-source, low-latency **Voice AI agent runtime and platform** — the engine that powers natural spoken conversations between a human and an AI agent over a phone-like channel. It streams live audio in, runs **Speech-to-Text** → **LLM orchestration** → **Text-to-Speech**, manages conversation state, memory and tool-calling, handles interruptions naturally, and streams audio back — with first-class **evaluation, observability, fault tolerance, and deployment**.

It is deliberately **not** a generic voice assistant or chatbot demo. It is built to read like the core of a real Voice AI startup product, and to produce concrete, defensible system-design and AI-engineering stories for interviews.

The entire system runs on a **100% free, open-source stack** on a single developer machine (Windows 11, NVIDIA GTX 1650 4 GB VRAM, 16 GB RAM), and every model/provider sits behind a clean interface so it can be swapped for a commercial provider (Deepgram, ElevenLabs, Cartesia, OpenAI, Anthropic, Sarvam) **without changing the architecture**.

Hero differentiators

1. **Turn-taking & barge-in engine** — VAD + semantic endpointing + predictive turn detection, plus natural interruption handling that cancels in-flight speech and reconciles state.
2. **Provider-agnostic, swappable architecture** — STT, LLM, TTS, VAD, turn-detector, and transport all behind interfaces, each with ≥ 2 implementations (local-OSS + cloud-stub).
3. **Built-in evaluation + observability suite** — synthetic-caller simulation, ASR-noise/adversarial injection, LLM-as-judge scoring, per-turn latency tracing, regression testing in CI.
4. **Pluggable transport** — browser (WebSocket/WebRTC) **and** simulated telephony (8 kHz, SIP-like lifecycle), behind one interface, with no paid carrier.
5. **Measured, optimized low latency** — an explicit, instrumented voice-to-voice latency budget with reported numbers.

2. Problem Statement & Motivation

Building a *demo* voice bot is easy; building a voice agent that feels **natural, fast, and reliable** at production quality is hard. The genuinely difficult engineering problems — the ones that distinguish a real Voice AI product from a tutorial — are:

- **Latency.** Humans perceive conversational lag above ~300–500 ms. A naive STT→LLM→TTS pipeline that waits for each stage to fully complete easily exceeds 3–5 seconds. Hitting sub-second response requires streaming at every stage.
- **Turn-taking & interruption (barge-in).** Knowing *when the user has finished speaking* (endpointing) and *gracefully stopping* when the user interrupts the agent is unsolved enough that it is an active research area at every serious voice company.
- **Robustness to ASR errors.** Real audio is noisy, accented, code-switched, and disfluent. The agent must stay coherent despite imperfect transcripts.

- **State, orchestration, memory, tools.** Real conversations branch, call backend functions, and remember context across turns.
- **Production concerns.** Observability, fault tolerance, evaluation, testing, and deployment are usually missing from portfolio projects — and are exactly what hiring managers probe in system-design interviews.

The target employers (Sarvam, Gnani, Bolna, Ring, Riverline, SquadStack) build **telephony contact-center voice agents** (sales, collections, support), often **multilingual including Indic languages and Hinglish code-switching**. Sutradhar is designed to demonstrate competence across exactly this surface area.

Why this is a strong resume artifact

It lets the candidate say, with evidence: *"I built a streaming voice-agent runtime with a custom turn-taking/barge-in engine, hit sub-1.2 s voice-to-voice latency on a 4 GB GPU, made every component provider-swappable, and shipped an evaluation + observability suite with regression testing in CI."* Every clause maps to a real interview discussion.

3. Goals & Non-Goals

3.1 Goals

- **G1** — A working, end-to-end, real-time streaming voice agent runnable locally on the target hardware.
- **G2** — A custom turn-taking + barge-in engine with measurable, natural interruption behavior.
- **G3** — Provider-agnostic interfaces with ≥ 2 implementations each (local-OSS + cloud-stub).
- **G4** — An explicit, instrumented latency budget with reported P50/P95 numbers.
- **G5** — Pluggable transport: browser client and simulated telephony.
- **G6** — Evaluation harness: synthetic callers, ASR-noise injection, LLM-as-judge, CI regression.
- **G7** — Observability: structured logs, distributed tracing, metrics dashboards.
- **G8** — Fault tolerance: failover, retries, circuit breakers, graceful degradation.
- **G9** — Deployment: Docker Compose stack, CI, documentation, ADRs, demo.
- **G10** — English-first quality with the model layer designed so Indic/Hinglish models drop in.

3.2 Non-Goals

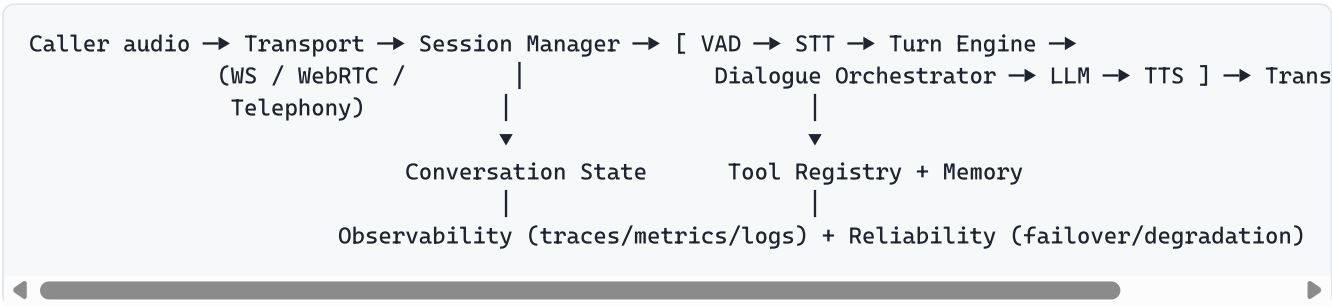
- **NG1** — Real PSTN/carrier telephony (Twilio/Exotel/SIP trunks) in the default path. We *simulate* telephony; a real adapter is documented as future work behind the same interface.
 - **NG2** — Training or fine-tuning custom speech/LLM models from scratch (we may fine-tune small endpointing classifiers, but base models are pretrained OSS).
 - **NG3** — A polished consumer product UI. The web client is a functional demo/diagnostic surface, not a design artifact.
 - **NG4** — Production-grade multi-tenant SaaS, billing, or auth. Architecture allows for it; it is out of scope for v1.
 - **NG5** — Guaranteeing commercial-grade Indic ASR/TTS quality on free models; we ensure the *pathway* exists.
-

4. Target Users & Personas

Persona	Description	What they need from Sutradhar
The reviewer (primary)	Hiring manager / founding engineer at a Voice AI company	Evidence of real streaming, latency, turn-taking, and production engineering depth
The developer	The author extending the platform	Clean interfaces, fast local dev loop, tests, docs
The end caller (simulated)	A person (or synthetic persona) talking to the agent	Natural, low-latency conversation; can interrupt; gets tasks done
The operator	Whoever runs the system	Dashboards, traces, health checks, graceful degradation

5. Product Overview & Vision

Sutradhar is structured as a **layered, async streaming pipeline** wrapped in production scaffolding:



Each pipeline stage is an independent async component communicating over **bounded queues with explicit backpressure and cancellation**. The **Conversation State Machine** is the single source of truth that barge-in reconciles against. Everything is configurable and swappable.

6. System Architecture

6.1 Architectural principles

- **Streaming-first.** Every stage emits incrementally (STT partials, LLM tokens, TTS audio chunks) so audio starts before the full response exists.
- **Interface-driven.** Components depend on abstractions (`Protocol` /ABC), never concrete providers.
- **Single source of truth.** The state machine owns turn state; components are stateless where possible.
- **Cancellable by design.** Every in-flight operation can be cancelled mid-stream (required for barge-in).
- **Observable by default.** Every turn produces a trace; every stage emits latency + outcome metrics.
- **Fail soft.** A component failure degrades the conversation; it does not crash the session.

6.2 Layered view

Layer	Responsibility	Key elements
Transport	Move audio frames in/out of a session	WebSocket, WebRTC (aiortc), Simulated Telephony
Session	Lifecycle of one conversation	Session manager, per-session isolation, resource cleanup
Pipeline	The streaming STT→LLM→TTS loop	VAD, STT, Turn engine, Orchestrator, LLM, TTS
Dialogue	Conversation intelligence	State machine, prompts, memory, tool registry
Observability	See inside the system	Tracing (OTel/Jaeger), metrics (Prometheus/Grafana), structured logs
Reliability	Keep running under failure	Failover, retries, circuit breakers, degradation, health
Eval	Measure quality and latency	Caller simulator, noise injection, LLM-judge, regression suite

6.3 Core interfaces

All under `sutradhar/interfaces/`. Each has **≥2 implementations**.

Interface	Methods (illustrative)	Local-OSS impl	Swap target (stubbed)
<code>Transport</code>	<code>recv_audio()</code> , <code>send_audio()</code> , <code>on_session_event()</code>	WebSocket, Telephony-sim	WebRTC, real SIP
<code>VAD</code>	<code>detect(frame) → SpeechProb</code>	Silero VAD	webrtcvad
<code>STT</code>	<code>stream(audio) → AsyncIterator[Partial/Final]</code>	faster-whisper	Vosk/Moonshine, Deepgram-stub
<code>TurnDetector</code>	<code>should_endpoint(state) → bool</code>	VAD+pause policy	semantic classifier, LiveKit-smart-turn-style
<code>LLM</code>	<code>stream(messages, tools) → AsyncIterator[Token/ToolCall]</code>	Ollama (Qwen2.5-3B)	OpenAI/Anthropic- compatible-stub
<code>TTS</code>	<code>stream(text) → AsyncIterator[AudioChunk]</code>	Piper	Kokoro, ElevenLabs/Cartesia-stub
<code>MemoryStore</code>	<code>append()</code> , <code>retrieve()</code>	SQLite + sqlite- vec	Chroma, Redis
<code>Tracer</code>	<code>span()</code> , <code>event()</code>	OpenTelemetry	—

7. Technology Stack (tuned for GTX 1650 / 4 GB VRAM / 16 GB RAM)

The 4 GB VRAM budget is the dominant constraint. Model placement is **deliberate**, not accidental: keep the GPU for the latency-critical STT model, run the LLM on CPU with partial GPU offload, and use a fast CPU TTS.

Concern	Choice	Placement	Rationale
Language/runtime	Python 3.11+, asyncio	—	Standard for voice AI; rich OSS ecosystem
STT	faster-whisper (CTranslate2), <code>small</code> / <code>base</code> , int8_float16	GPU	Fast, accurate, low VRAM (~0.5–1 GB); streams partials/finals
VAD	Silero VAD (ONNX)	CPU	Tiny, accurate, low-latency
Turn detection	Silero pause logic + lightweight semantic endpoint classifier	CPU	Predictive endpointing without big VRAM cost
LLM	Ollama serving Qwen2.5- 3B-Instruct (or Llama-3.2- 3B), Q4_K_M	CPU + partial GPU offload	7B won't fit in 4 GB alongside STT; 3B streams tokens fast on CPU with 16 GB RAM; supports tool-calling
TTS	Piper (ONNX) default; Kokoro-82M optional	CPU	Piper is extremely fast (low first-audio latency); Kokoro for higher quality
Memory	SQLite + sqlite-vec (or Chroma)	CPU	Embedded, zero-ops, retrievable memory
Transport	WebSocket (default) + aiortc WebRTC + telephony-sim	CPU	Robust on Windows; WebRTC optional
Tracing	OpenTelemetry + Jaeger	Docker	Per-turn distributed traces
Metrics	Prometheus + Grafana	Docker	Latency/throughput dashboards
Logs	structlog (JSON)	—	Structured, queryable
Mock backend	FastAPI + SQLite	CPU	Demo vertical tool execution
Packaging	uv or Poetry, ruff, black, mypy, pre-commit	—	Clean DX
CI	GitHub Actions	—	Lint, test, eval regression
Experimentation	Google Colab (optional)	Cloud	Trial larger models; not in the local run path

Indic-ready note: STT/LLM/TTS interfaces are language-agnostic. English models are the default for reliability; an Indic/Hinglish profile (e.g., Indic-tuned Whisper, an Indic LLM, Indic Piper/Kokoro voices) drops in via config without touching the pipeline.

8. Latency Budget & Performance Requirements

Metric of record: *voice-to-voice latency* = time from end-of-user-speech (endpoint detected) to **first agent audio byte** emitted.

Stage	Target (P50)	Notes / optimization
VAD endpoint decision	50–100 ms	Frame-level, runs continuously
STT finalization after endpoint	150–250 ms	Most transcription already streamed as partials
LLM first token	300–600 ms	CPU 3B; prompt kept short; KV cache warm; streaming
TTS first audio chunk	100–300 ms	Piper; synthesize first clause only
Transport/buffering	30–80 ms	Small jitter buffer
Total voice-to-voice (P50 target)	≈ 0.8–1.2 s	Honest target for CPU-LLM on 4 GB; push lower where possible

Performance requirements

- **PR1** — Stream STT partials within ≤ 300 ms of speech onset.
- **PR2** — Begin TTS playback before the full LLM response completes (first-clause synthesis).
- **PR3** — Barge-in: stop agent audio within ≤ 200 ms of detecting user speech.
- **PR4** — Sustain ≥ 1 real-time conversation on the target machine; document concurrency ceiling.
- **PR5** — Every stage's latency is recorded per turn and visible in Grafana.

9. Turn-Taking & Barge-In (Hero Feature #1)

9.1 Endpointing (when did the user stop?)

A **hybrid policy**, pluggable behind `TurnDetector` :

- **Acoustic:** Silero VAD detects speech/silence; a configurable trailing-silence threshold proposes an endpoint.
- **Semantic:** a lightweight classifier judges whether the current transcript is a *complete* utterance (reduces premature cut-offs on natural pauses, and reduces lag on clearly-finished sentences). This is the "predictive turn detection" element.
- **Policy fusion:** combine acoustic + semantic signals with tunable thresholds; expose metrics on false-early and false-late endpoints.

9.2 Barge-in (user interrupts the agent)

State-machine driven, with **clean cancellation**:

1. While the agent is in `SPEAKING`, VAD continuously monitors the inbound channel.
2. On confirmed user speech, the engine **immediately**: (a) stops TTS playback, (b) cancels in-flight LLM + TTS tasks via cancellation tokens, (c) flushes output buffers.
3. The state machine records *what was actually spoken* (truncated agent turn) so context stays accurate.
4. Transition `SPEAKING → INTERRUPTED → LISTENING`; begin transcribing the user's new input.
5. No state corruption: the conversation history reflects the real, truncated exchange.

9.3 Turn states

`IDLE → LISTENING → THINKING → SPEAKING → (INTERRUPTED) → LISTENING ...`

Each transition is logged, traced, and unit-tested. Barge-in correctness is a first-class eval metric.

10. Conversation State, Memory & Tool-Calling

- **State machine** (`dialogue/state.py`): owns turn state, the rolling message history, and the active tool-call cycle. Single source of truth.
- **Memory**:
 - *Short-term*: rolling window of recent turns within the LLM context budget.
 - *Long-term*: SQLite + vector store; retrieve relevant prior facts (caller details, prior outcomes) and inject into the prompt.
- **Tool-calling**: a `ToolRegistry` of functions with JSON schemas; the LLM emits tool calls, the orchestrator executes them (against the mock backend), and feeds results back into the stream — all while keeping the conversation responsive.
- **Demo vertical (one, done well): outbound lead qualification / appointment booking / order-status support** (final pick at M3). Backed by a FastAPI + SQLite mock CRM with tools like `lookup_customer`, `book_slot`, `get_order_status`, `update_disposition`.

11. Evaluation Framework (Hero Feature #3)

Capability	Description
Synthetic caller simulator	An LLM-driven caller persona generates user turns; turns are rendered to speech (TTS) and fed through the real pipeline, producing transcripts + traces. Personas vary (cooperative, terse, interrupting, off-topic).
ASR-noise / adversarial injection	Inject background noise, clipping, disfluencies, accents, and code-switching into caller audio to test robustness to ASR errors.
LLM-as-judge scoring	Automated scoring of task success, coherence, instruction-following, and barge-in correctness , with rubric prompts.

Capability	Description
Latency evaluation	Objective per-turn latency capture (P50/P95) across scenarios.
Regression suite	A scenario library run in CI; the build fails on latency or quality regressions beyond thresholds.

12. Observability & Monitoring

- **Tracing:** OpenTelemetry spans per turn and per stage (VAD→STT→LLM→TTS), exported to **Jaeger**. A single turn is one trace; barge-in events are span events.
- **Metrics:** Prometheus counters/histograms for stage latency, tokens/sec, endpoint accuracy, barge-in count, error rates, active sessions — visualized in **Grafana** dashboards (shipped as JSON).
- **Logging:** `structlog` JSON logs with session/turn IDs for correlation.
- **Health:** `/healthz` and `/readyz` endpoints; component health surfaced to the operator.

13. Fault Tolerance & Reliability (Hero Feature #2 supports this)

- **Provider failover:** if the primary STT/LLM/TTS provider errors or times out, automatically fail over to a secondary implementation behind the same interface.
- **Retries with backoff** on transient failures; **circuit breakers** to stop hammering a dead provider.
- **Timeouts** at every stage with sensible defaults.
- **Graceful degradation:** if TTS fails, fall back to a simpler voice / canned audio; if the LLM stalls, emit a holding phrase; never drop the call silently.
- **Per-session isolation:** one session's failure cannot take down others; bounded resources per session.
- **Recovery:** sessions clean up resources deterministically on completion or failure.

14. Testing Strategy

Level	Scope	Tooling
Unit	Each component, state transitions, cancellation logic	pytest, pytest-asyncio
Integration	Full pipeline on recorded audio; provider swaps; barge-in scenarios	pytest
Eval/regression	Scenario suite + LLM-judge + latency thresholds	custom harness in CI
Load	Many concurrent simulated callers; find the concurrency ceiling	locust or custom async harness
Static	Types, lint, format	mypy, ruff, black, pre-commit

15. Deployment & Developer Experience

- **Docker Compose** stack: app, Ollama, Jaeger, Prometheus, Grafana, mock-backend. One command to bring up.
- **Windows-friendly**: documented setup for Windows 11; no Linux-only assumptions in the default path.
- **Makefile / task runner**: `setup` , `run` , `test` , `eval` , `demo` , `up` , `down` .
- **Config**: `pydantic-settings` + `.env` ; switch models/providers/latency targets without code changes.
- **Docs**: README with architecture diagrams, a latency report, a "swap a component" guide, and **ADRs** capturing key trade-offs.
- **One-command demo** + a short demo script.

16. Repository Structure

```
sutradhar/  
  core/           # pipeline, session manager, state machine, queues, cancellation  
  interfaces/     # ABCs/Protocols: Transport, VAD, STT, TurnDetector, LLM, TTS, Memory, Tools,  
  providers/      #  
    stt/          # faster-whisper, vosk/moonshine, cloud-stub  
    tts/          # piper, kokoro, cloud-stub  
    llm/          # ollama, openai-compatible-stub  
    vad/          # silero, webrtcvad  
    turn/         # acoustic+semantic endpointing  
  transport/      # websocket/, webrtc/, telephony/  
  dialogue/       # orchestrator, state machine, tool registry, memory, prompts  
  observability/  # tracing, metrics, logging  
  reliability/    # failover, circuit breaker, retries, degradation, health  
  eval/           # simulator, noise injection, judges, scenario suite  
  clients/web/    # browser mic client  
  mock_backend/   # FastAPI + SQLite demo vertical  
  deploy/        # docker, compose, grafana dashboards, prometheus config  
  tests/         # unit, integration, load  
  docs/          # architecture, ADRs, latency report, THIS PRD
```

17. Roadmap & Milestones

Flagship scope, ~2–5 weeks. Each milestone has deliverables and **acceptance criteria**; complete and verify before advancing. Maintain a living TODO and CHANGELOG.

Milestone	Deliverable	Acceptance criteria
M0 — Scaffold	Repo, packaging, config, interfaces, logging/metrics/tracing skeleton, CI, README	<code>make setup</code> works on Windows; CI green; interfaces defined
M1 — Core loop	Half-duplex streaming: mic→VAD→STT→LLM→TTS→speaker via browser WS client; latency instrumentation	A spoken question gets a spoken answer; per-stage latency recorded; baseline reported

Milestone	Deliverable	Acceptance criteria
M2 — Turn-taking & barge-in	Semantic endpointing + interruption cancellation + state reconciliation	User can interrupt; agent stops ≤ 200 ms; no state corruption; metrics emitted
M3 — Dialogue	State machine, memory, tool-calling, demo vertical + mock backend	Multi-turn task completes via tool calls; memory persists across turns
M4 — Evaluation	Synthetic callers, ASR-noise injection, LLM-judge, CI regression	<code>make eval</code> runs scenarios; regression gate in CI
M5 — Transport & reliability	Telephony-sim adapter; failover, degradation; load test	Same agent runs over telephony-sim; kills a provider and conversation survives; concurrency ceiling documented
M6 — Ops & polish	Grafana dashboards, Docker Compose, docs/ADRs/latency report, demo	<code>docker compose up</code> brings full stack; dashboards live; demo script runs

18. Functional Requirements

- **FR1** — Accept streaming audio from a browser client over WebSocket.
- **FR2** — Detect speech and endpoints using a pluggable VAD + turn detector.
- **FR3** — Transcribe streaming audio, emitting partial and final results.
- **FR4** — Orchestrate an LLM with conversation history, memory, and tool-calling, streaming tokens.
- **FR5** — Synthesize streaming TTS audio, beginning before the full response is generated.
- **FR6** — Detect and handle barge-in, cancelling in-flight work and reconciling state.
- **FR7** — Persist and retrieve short- and long-term conversation memory.
- **FR8** — Execute tool calls against the mock backend and feed results back into the conversation.
- **FR9** — Run the same agent over a simulated-telephony transport.
- **FR10** — Run an evaluation suite producing transcripts, scores, and latency reports.
- **FR11** — Emit traces, metrics, and structured logs for every turn.
- **FR12** — Fail over between provider implementations on error/timeout.
- **FR13** — Allow swapping any provider via configuration without code changes.

19. Non-Functional Requirements

- **NFR1 (Latency)** — Voice-to-voice P50 $\approx 0.8\text{--}1.2$ s on target hardware; reported and tracked.
- **NFR2 (Modularity)** — All providers behind interfaces; ≥ 2 implementations each.
- **NFR3 (Cost)** — 100% free/OSS in the default run path.
- **NFR4 (Portability)** — Runs locally on Windows 11 / GTX 1650 4 GB / 16 GB RAM.
- **NFR5 (Observability)** — Every turn traced; key metrics dashboarded.

- **NFR6 (Reliability)** — No single component failure crashes a session.
- **NFR7 (Maintainability)** — Typed, linted, tested; ADRs document decisions.
- **NFR8 (Extensibility)** — Commercial-provider swap requires only a new adapter + config.
- **NFR9 (Internationalization)** — Model layer supports dropping in Indic/Hinglish models.

20. Risks & Mitigations

Risk	Impact	Mitigation
4 GB VRAM can't fit STT + LLM together	High	LLM on CPU + partial offload; STT on GPU; measured placement; Colab only for experimentation
CPU LLM latency too high	Med	Small 3B model, short prompts, KV-cache warmth, token streaming, first-clause TTS
WebRTC (aiortc) flaky on Windows	Med	WebSocket transport is the default; WebRTC is optional behind the interface
Barge-in causes state corruption	High	State machine as single source of truth; cancellation tokens; dedicated tests
Free Indic ASR/TTS quality weak	Med	English-first default; Indic is a documented, config-swappable profile
Scope creep (flagship is large)	Med	Strict milestone gating; each milestone independently demoable
Eval LLM-judge unreliable	Med	Rubric prompts, multiple judge runs, human spot-checks

21. Success Metrics / KPIs

- **K1** — Voice-to-voice P50 latency ≤ 1.2 s; P95 reported.
- **K2** — Barge-in stop time ≤ 200 ms; barge-in correctness $\geq 95\%$ on the scenario suite.
- **K3** — Endpoint accuracy (false-early + false-late rate) tracked and improved across milestones.
- **K4** — ≥ 2 implementations live for STT, LLM, TTS, Transport.
- **K5** — Eval regression gate active in CI; ≥ 10 scenarios.
- **K6** — Full stack `docker compose up` works; dashboards populated.
- **K7** — Provider kill test: conversation survives a primary-provider failure.

22. Future Work (commercial path)

- Real telephony via SIP/Twilio/Exotel adapters behind `Transport`.
- Commercial provider adapters (Deepgram, ElevenLabs, Cartesia, OpenAI, Anthropic, Sarvam) — already stubbed.
- Full-duplex conversation (overlapping speech) and speech-to-speech models (e.g., Moshi-style).

- Multi-tenant SaaS, auth, billing, autoscaling.
- Production Indic/Hinglish profiles with quality benchmarks.

23. Appendix

A. Glossary

- **STT/ASR** — Speech-to-Text / Automatic Speech Recognition.
- **TTS** — Text-to-Speech.
- **VAD** — Voice Activity Detection.
- **Endpointing** — Deciding when the user has finished their turn.
- **Barge-in** — The user interrupting the agent while it speaks.
- **Voice-to-voice latency** — End-of-user-speech to first agent audio.
- **LLM-as-judge** — Using an LLM to score conversation quality against a rubric.

B. Default model table

Role	Default (free/OSS)	Placement	Swap target
STT	faster-whisper small/base int8	GPU	Deepgram / Sarvam
LLM	Qwen2.5-3B-Instruct (Ollama, Q4)	CPU+offload	GPT/Claude/Sarvam
TTS	Piper (Kokoro optional)	CPU	ElevenLabs / Cartesia
VAD	Silero VAD	CPU	webrtcvad

C. Key references / inspiration

- Streaming voice-agent frameworks: Pipecat, LiveKit Agents, Bolna (OSS), Vocode.
- Endpointing/turn-detection research and "smart-turn"-style models.
- faster-whisper / CTranslate2, Silero VAD, Piper, Kokoro, Ollama, Qwen2.5 / Llama 3.2.